

Задача:

На языке программирования C/C++, для любого микроконтроллера, реализовать нижеописанный протокол.

Интерфейс - UART.

Использование RTOS обязательно.

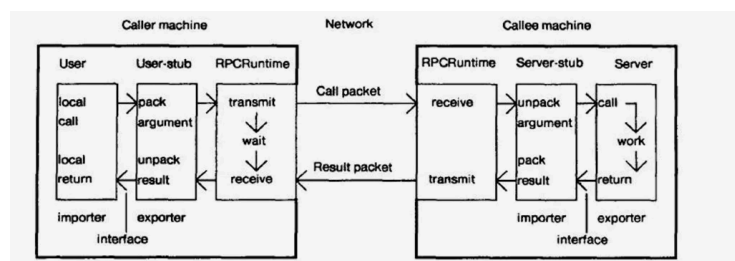
Реальный запуск на устройстве не требуется, достаточно собрать проект.

К ответу приложить исходный код на github. Приложить описание реализации.

Сделать предложения по улучшению протокола. Рассмотреть недостатки протокола.

Описание протокола:

Протокол реализует упрощенную версию технологии RPC (Remote Procedure Call).



В контексте протокола его можно разделить на 4 части:

- Уровень приложения
- Транспортный уровень
- Канальный уровень
- Физический уровень

1. Физический уровень

Физический уровень отвечает за непосредственную отправку последовательности байт в некоторую среду передачи данных. Эта среда должна быть асинхронной, либо поддерживать асинхронную обертку.

Для каждой среды реализация физического уровня является уникальной.

2. Канальный уровень

Канальный уровень отвечает за формирование пакетов при передаче и разделение входящего потока байт на пакеты при приеме. Для проверки целостности данных используется контрольная сумма.

Формат пакета для канального уровня:

№ байта	0	1	2	3	4	5...n
Значение	0xFA	l l	l h	crc8	0xFB	x
Расшифровка	стартовый байт	младший байт длины пакета	старший байт длины пакета	контрольная сумма заголовка	стартовый байт данных	сообщение транспортного уровня

№ байта	n+1	n+2
Значение	crc8	0xFE
Расшифровка	контрольная сумма всего пакета	стоповый байт

3. Транспортный уровень

Транспортный уровень при приеме отвечает за расшифровку входящих пакетов, выполнения связанной функции и отправку результата, при передаче выполняется проверка таймаута ответа другой стороной.

Формат сообщения транспортного уровня:

№ байта	0	1	2...n	n+1	n+2...k
Значение	0x0B/0x0C/0x16/0x21	N	name	0	x
Расшифровка	тип сообщения (запрос/стрим/ответ/ошибка)	порядковый номер сообщения	текстовое название функции	терминатор (разделительный 0)	аргументы вызова функции в байтовой форме

Логика работы транспортного уровня при отправке:

- Сформировать запрос и передать его на канальный уровень
- Дождаться ответа на запрос (опрашивая очередь приема), в качестве идентификатора используется порядковый номер
- Вернуть данные ответа на запрос в случае успеха, иначе вернуть ошибку

Логика работы транспортного уровня при приеме:

- Постоянно проверять наличие запросов от канального уровня (это включает в себя вызов функций приема канального и физического уровней)
- При поступлении ответа на запрос – передать его в очередь приема.
- При поступлении запроса – выполнить поиск среди зарегистрированных функций, если функция не найдена возврат ошибки. В случае нахождения функции – необходимо вызвать ее с аргументом в виде указателя на принятую полезную нагрузку. Вызываемая функция должна вернуть результат до истечения таймаута.
- Отправить сообщение «ошибка», либо «ответ на запрос» - с полезной нагрузкой, представленной вызванной функцией.

4. Уровень приложения

Уровень приложения – содержит код верхнего уровня, с этим кодом работает программист.

Задачи этого уровня:

- Зарегистрировать перечень функций доступных для удаленного вызова
- Создать задачу приема сообщений.
- Обеспечить возможность ожидания ответа на отправленный запрос.